

DMATH-ALGO

Rekursive Algorithmen

Herbstsemester 24

Serie 6

Lernziele

- Sie verstehen wie rekursive Algorithmen arbeiten und können Rekursionsbäume zeichnen.
- Sie können die Komplexität für divide-and-conquer Algorithmen mit Big- Θ abschätzen.
- Sie können lineare Rekursionsbeziehungen mit konstanten Koeffizienten auflösen.

Die nicht-rekursiven Algorithmen in der letzten Serie konnten durch Zählen und Manipulieren von Summen analysiert werden. Zunächst haben wir die Anzahl der Iterationen jeder Schleife ermittelt und dann herausgefunden, wie lange jede Iteration dauert. Die Bestimmung der Laufzeit eines rekursiven Algorithmus ist schwieriger. Er enthält nicht nur Schleifen - die Laufzeit des Algorithmus für eine Eingabe der Grösse n hängt von der Laufzeit desselben Algorithmus für Eingaben mit einer Grösse kleiner als n ab.

Frage 1: Was ist ein rekursiver Algorithmus?

Eine Methode zur Problemlösung, bei der eine Funktion sich selbst aufruft, um kleinere Instanzen desselben Problems zu bearbeiten, nennen wir einen **rekursiven Algorithmus**. Diese Art von Algorithmen ist besonders interessant bei Aufgaben, die sich leicht in identische, kleinere Probleme unterteilen lassen:

- Eine Funktion kann für kleinen Input, etwa für $n = 1$ effizient direkt berechnet werden.
- Für einen grossen Input-Wert n teilen wir das Problem in gleiche Probleme mit kleinerem Input-Wert und bestimmen deren Lösung.
- Dann rekonstruieren wir aus diesen Teillösungen den gesuchten Output-Wert für n .

Solche Verfahrenen heissen **divide-and-conquer** Algorithmen.

Aufgabe 1: Bestimmen Sie rekursive Algorithmen zur Berechnung der **Summe** von den ersten n natürlichen Zahlen und zur Berechnung der **Fakultät** $n!$. Zeichnen Sie je einen **Rekursionsbaum** für $n = 5$.

$1 + 2 + 3 + 4 + 5 = (1 + 2 + 3 + 4) + 5$

```
def summe(n):  
    if n == 1:  
        return 1  
    else:  
        return summe(n-1) + n
```

summe(5) = 10 + 5 = 15
↓
summe(4) = 6 + 4 = 10
↓
summe(3) = 3 + 3 = 6
↓
summe(2) = 1 + 2 = 3
↓
summe(1)

$1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = (1 \cdot 2 \cdot 3 \cdot 4) \cdot 5$

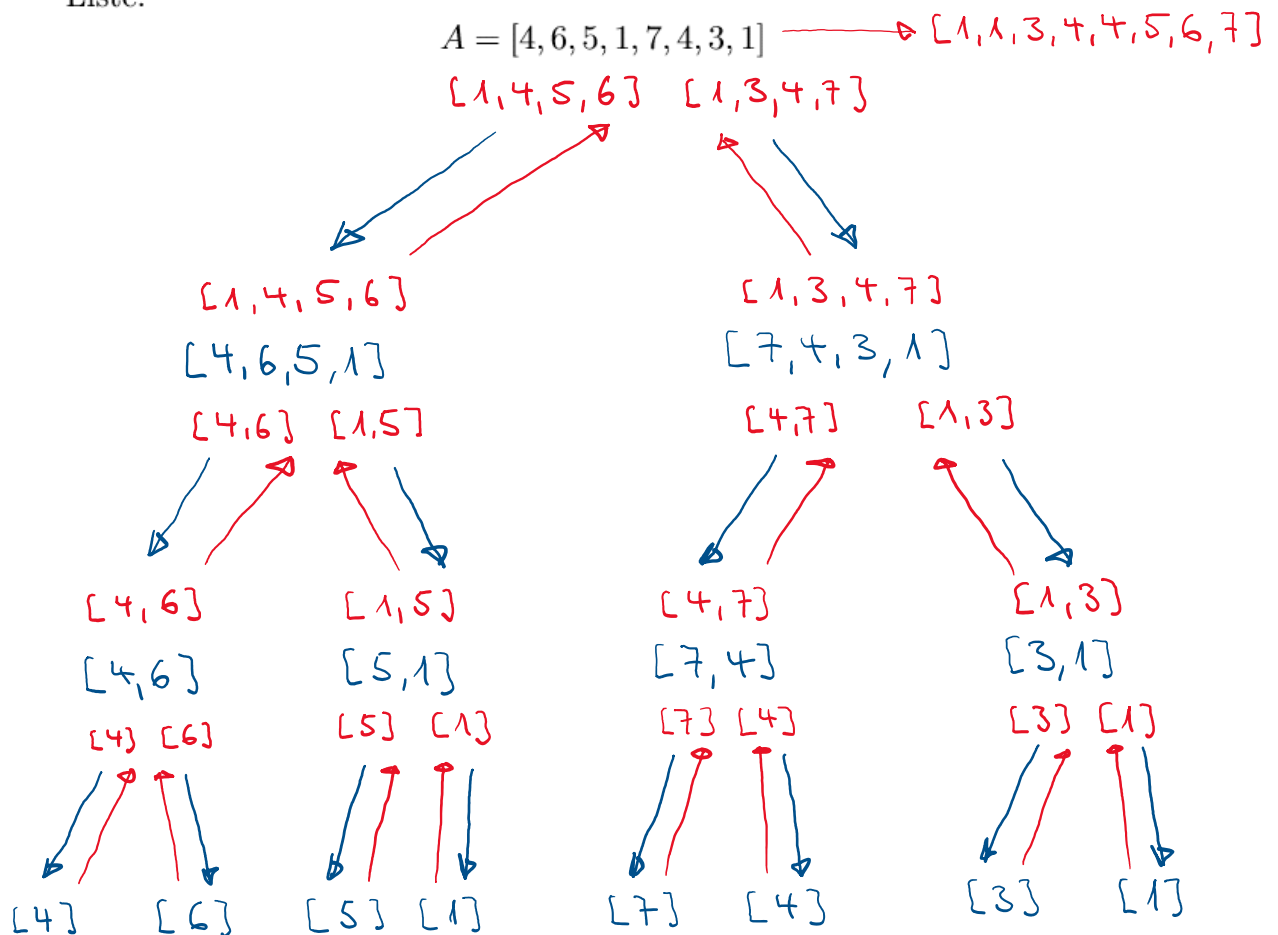
```
def fakultät(n):  
    if n == 1:  
        return 1  
    else:  
        return fakultät(n-1) * n
```

fakultät(5) = 24 * 5 = 120
↓
fakultät(4) = 6 * 4 = 24
↓
fakultät(3) = 2 * 3 = 6
↓
fakultät(2) = 1 * 2 = 2
↓
fakultät(1)

Der ungarisch-US-amerikanische Mathematiker John von Neumann (1903-1957) hat 1945 erstmals einen sehr effizienten Sortieralgorithmus vorgestellt, der sogenannte *mergeSort*-Algorithmus:

```
def mergeSort(A):
    if len(A)==1:
        return A
    L = mergeSort([A[i] for i in range(len(A)//2)])
    R = mergeSort([A[i] for i in range(len(A)//2,len(A))])
    return merge(L,R)
```

Aufgabe 2: Zeichnen Sie einen *Rekursionsbaum* für die folgende Input-Liste:



Vermutlich wurde das Spiel *Türme von Hanoi* 1883 vom französischen Mathematiker Édouard Lucas erfunden. Er dachte sich dazu die Geschichte aus, dass indische Mönche im grossen Tempel zu Benares, im Mittelpunkt der Welt, einen Turm aus 64 goldenen Scheiben versetzen müssten, und wenn ihnen das gelungen sei, wäre das Ende der Welt gekommen.

Ziel des Spieles: Alle Scheiben vom Turm in der Mitte (im Bild sind es acht Scheiben) sollen zu einem Turm über dem Pfosten ganz rechts bewegt werden. Bedingungen: Es darf nur eine Scheibe pro Zug verschoben werden, eine grössere Scheibe darf nie auf einer kleineren Scheibe liegen. Ein Zug besteht aus dem umlegen einer Scheibe von einem Pfosten zu einem anderen Pfosten.



Aufgabe 3: Entwickeln Sie einen rekursiven Algorithmus in Pseudocode, der diese Aufgabe löst. Implementieren Sie diesen Algorithmus in einer Programmiersprache ihrer Wahl (optional).

HanoiSolver (n Scheiben von Mitte nach rechts)

 wenn $n == 1$:

 1 Scheibe von Mitte nach rechts

 sonst:

 HanoiSolver (n-1 Scheiben von Mitte nach links)

 1 Scheibe von Mitte nach rechts

 HanoiSolver (n-1 Scheiben von links nach rechts)

Frage 2: Wie werden rekursive Algorithmen analysiert?

Für die totale Anzahl Rechnungsschritte eines rekursiven Algorithmus können wir ebenfalls eine rekursive Berechnung formulieren. Um dann eine Big- Θ Abschätzung für die Komplexität zu machen, müssen wir diese rekursive Berechnung auflösen.

Aufgabe 4: Bestimmen Sie für die rekursive Berechnung der Summe und der Fakultät eine Big- Θ Abschätzung:

$T(n)$ = Anzahl Rechnungsschritte für Input n

```
def summe(n):
    if n == 1: ①
        return 1 ①
    else:
        return summe(n-1) + n ①
```

$T(n-1)$

```
def faktätät(n):
    if n == 1: ①
        return 1 ①
    else:
        return faktätät(n-1) * n ①
```

$T(n-1)$

$summe(n)$ und $faktätät(n)$ brauchen gleichviele Rechnungsschritte:

$$\begin{aligned} T(1) &= 2 \\ T(n) &= 1 + T(n-1) + 2 \\ &= 3 + T(n-1) \end{aligned}$$

Rekursion auflösen:

$$T(1) = 2$$

$$T(2) = 3 + T(1) = 3 + 2$$

$$T(3) = 3 + T(2) = 3 + (3 + 2) = 3 + 3 + 2$$

$$T(4) = 3 + T(3) = 3 + (3 + 3 + 2) = \underbrace{3 + 3 + 3 + 2}_{3 \text{ mal}}$$

$\vdots$

$$T(n) = \underbrace{3 + \dots + 3}_{(n-1) \text{ mal}} + 2 = (n-1) \cdot 3 + 2 = \underline{\underline{3n-1}}$$

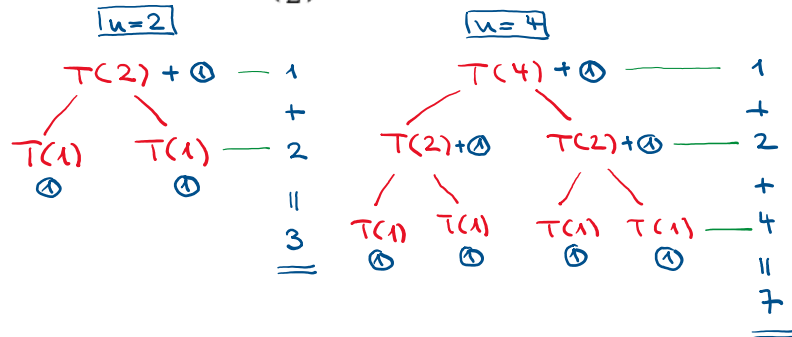
$$\left. \begin{aligned} T(n) &\leq 3n - 0 = 3 \cdot n \text{ für } n \geq 1 \\ T(n) &\geq 3n - n = 2 \cdot n \text{ für } n \geq 1 \end{aligned} \right\} T(n) \text{ ist in } \Theta(n)$$

Seien A , B und C drei rekursive Algorithmen.

Aufgabe 5: Bestimmen Sie eine Big- Θ Abschätzung für die Komplexität dieser Algorithmen, wenn die Berechnung für ihre totale Anzahl Rechnungsschritte im Worst-Case durch folgende rekursive Beziehungen gegeben sind:

Hinweis: Studieren Sie den Rekursionsbaum für einen Input-Wert der Form $n = 2^k$.

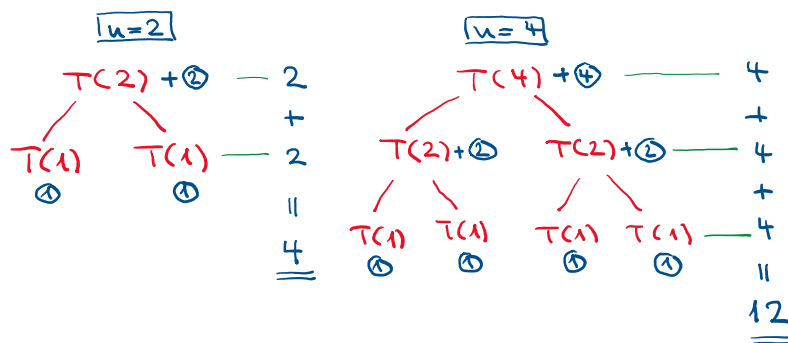
◦ $T_A(n) = 2 \cdot T_A\left(\frac{n}{2}\right) + 1$ und $T_A(1) = 1$



n	$T(n)$
2	$1+2=3$
4	$3+4=7$
8	$7+8=15$
16	$15+16=31$
⋮	⋮
n	$(n-1)+n=2n-1$

in $\Theta(n)$

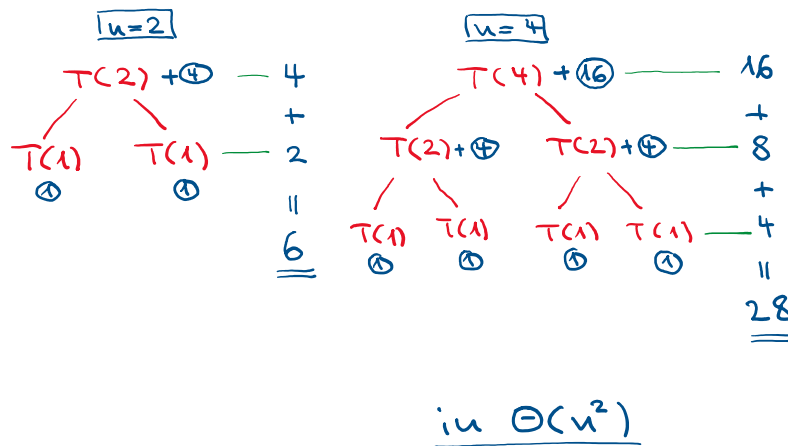
◦ $T_B(n) = 2 \cdot T_B\left(\frac{n}{2}\right) + n$ und $T_B(1) = 1$



n	$T(n)$
$2^1=2$	$2 \cdot 2 = 4$
$2^2=4$	$3 \cdot 4 = 12$
$2^3=8$	$4 \cdot 8 = 32$
$2^4=16$	$5 \cdot 16 = 80$
⋮	⋮
n	$(\log_2 n + 1) \cdot n$

in $\Theta(n \log n)$

◦ $T_C(n) = 2 \cdot T_C\left(\frac{n}{2}\right) + n^2$ und $T_C(1) = 1$



n	$T(n)$
2	$2+4=2 \cdot (1+2)$
4	$4+8+16=4 \cdot (1+2+4)=4 \cdot (3+4)$
8	$8+16+32+64=8 \cdot (1+2+4+8)=8 \cdot (7+8)$
⋮	⋮
n	$n \cdot (n-1+n) = n(2n-1) = 2n^2 - n$

Beim mergeSort Algorithmus finden wir für die totale Anzahl Rechenschritte im Worst-Case die rekursive Berechnung

$$T(n) = T\left(\left\lceil \frac{n}{2} \right\rceil\right) + T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + \text{Anzahl Rechnungsschritte für merge}$$

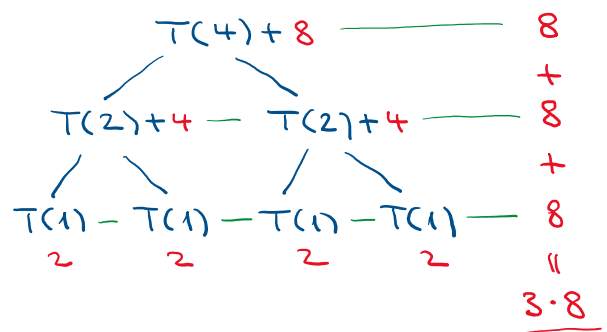
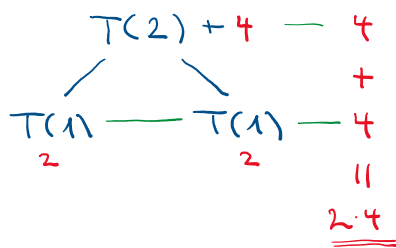
Aufgabe 6: Zeigen Sie, dass mergeSort in $\Theta(n \log(n))$ ist.

Hinweis: Studieren Sie den Rekursionsbaum von mergeSort für eine Liste mit einer Anzahl Elemente der Form $n = 2^k$.

Anzahl Rechnungsschritte für **merge**:
Wir müssen zwei sortierte Listen der Länge $n/2$ zu einer sortierten Liste der Länge n zusammenfügen. z.B. für **$n=6$** :

$[1, 3, 5] [2, 4, 6] [] \rightarrow [3, 5] [2, 4, 6] [3]$
 $\quad \quad \quad \underbrace{\quad \quad \quad}_{1 \text{ oder } 2?}$
 6 mal einen Test machen
 +
 6 mal ein Element verschieben
 = 2 · 6 Schritte
 allgemein $2n$ Schritte für merge.

$\rightarrow [3, 5] [4, 6] [1, 2]$
 $\quad \quad \quad \underbrace{\quad \quad \quad}_{3 \text{ oder } 2?}$
 $\rightarrow [5] [4, 6] [1, 2, 3]$
 $\quad \quad \quad \underbrace{\quad \quad \quad}_{3 \text{ oder } 4?}$
 $\rightarrow [5] [6] [1, 2, 3, 4]$
 $\quad \quad \quad \underbrace{\quad \quad \quad}_{5 \text{ oder } 6?}$
 $\rightarrow [] [6] [1, 2, 3, 4, 5]$
 $\quad \quad \quad \underbrace{\quad \quad \quad}_{* \text{ oder } 6?}$
 $\rightarrow [] [] [1, 2, 3, 4, 5, 6]$



für $n = 2^k$ gilt:

$$T(n) = \underbrace{2n + 2n + \dots + 2n}_{k+1 \text{ mal}} = (k+1) \cdot 2n = (\lfloor \log_2(n) \rfloor + 1) \cdot 2n$$

ist in $\Theta(\log(n) \cdot n)$

Rekursive Beziehungen wie in der Aufgabe 6 kommen in der Informatik häufig vor, weil sie die Rechnungsschritte von divide-and-conquer Algorithmen beschreiben. Allgemein haben diese Beziehungen die Form

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + c \cdot n^k \quad \text{und} \quad T(1) = c.$$

Die Zahl a gibt an, wie viele verschiedene rekursive Aufrufe mit einem Input-Wert der Grösse $\frac{n}{b}$ gemacht werden. Mit $c \cdot n^k$ wird beschrieben, wie viele Rechenschritte nötig sind, um das Problem in kleinere Teilprobleme zu zerlegen und dann die Lösung für den Input-Wert n aus den Teillösungen zu rekonstruieren.

Satz: Für konstante Zahlen $a \geq 1$, $b > 1$, $c > 0$ und $k \geq 0$ gilt:

- Wenn $b^k < a$ ist, dann ist $T(n)$ in $\Theta(n^{\log_b(a)})$.
- Wenn $b^k = a$ ist, dann ist $T(n)$ in $\Theta(n^k \log(n))$.
- Wenn $b^k > a$ ist, dann ist $T(n)$ in $\Theta(n^k)$.

Aufgabe 7: Bestimmen Sie mit diesem Satz eine Big- Θ Abschätzung für die Komplexität eines Algorithmus, dessen totale Anzahl Rechnungsschritte im Worst-Case durch folgende rekursive Beziehung gegeben ist:

- a) $T(n) = 4 \cdot T\left(\frac{n}{3}\right) + n^2$ und $T(1) = 1$. $a=4, b=3, k=2$
 $b^k = 3^2 = 9 > 4 = a$ also $\Theta(n^2)$
- b) $T(n) = 2 \cdot T\left(\frac{n}{3}\right) + n^4$ und $T(1) = 1$. $a=2, b=3, k=4$
 $b^k = 3^4 = 81 > 2 = a$ also $\Theta(n^4)$
- c) $T(n) = 16 \cdot T\left(\frac{n}{4}\right) + n^2$ und $T(1) = 1$. $a=16, b=4, k=2$
 $b^k = 4^2 = 16 = a$ also $\Theta(n^2 \log(n))$
- d) $T(n) = 4 \cdot T\left(\frac{n}{2}\right) + 1$ und $T(1) = 1$. $a=4, b=2, k=0$
 $b^k = 2^0 = 1 < 4 = a$ also $\Theta(n^2)$ $\log_2(4)=2$
- e) $T(n) = 4 \cdot T\left(\frac{n}{4}\right) + n$ und $T(1) = 1$. $a=4, b=4, k=1$
 $b^k = 4^1 = 4 = a$ also $\Theta(n \log(n))$

Frage 3: Was sind lineare Rekursionsbeziehungen?

Eine weitere Sorte von Rekursionsbeziehungen kommen in vielen Modellen der Informatik vor.

Definition 1: Eine Rekursionsbeziehung der Form

$$T(n) = c_1 \cdot T(n-1) + c_2 \cdot T(n-2) + \dots + c_k \cdot T(n-k) + F(n)$$

heiss **linear** vom **Grad** k mit **konstanten Koeffizienten** $c_1, c_2, \dots, c_k$.

- Wenn $F(n) = 0$ ist, heisst die Beziehung **homogen**.
- Wenn $F(n) \neq 0$ ist, heisst die Beziehung **inhomogen**.

Satz 1: Für die Auflösung einer **homogenen** Rekursionsbeziehung gilt

- für den Grad **$k = 1$** finden wir die Lösung direkt:
 - **$T(n) = a \cdot c_1^n$**
- für den Grad **$k = 2$** müssen wir die Lösungen r_1 und r_2 der quadratischen Gleichung $r^2 = c_1 \cdot r + c_2$ bestimmen:
 - Wenn $r_1 \neq r_2$, dann gilt **$T(n) = a \cdot r_1^n + b \cdot r_2^n$**
 - Wenn $r_1 = r_2$, dann gilt **$T(n) = a \cdot r_1^n + b \cdot n r_1^n$**

mit Konstanten a und b . Diese Konstanten können wir aus den Anfangsbedingungen der Rekursionsbeziehung explizit bestimmen.

Satz 2: Für die Auflösung einer **inhomogenen** Rekursionsbeziehung gilt:

- Bestimme zuerst die allgemeine Lösung **$T_h(n)$** der homogenen Rekursionsbeziehung nach Satz 1, ohne den Term $F(n)$.
- Suche eine einzige Lösung **$T_p(n)$** der inhomogenen Rekursionsbeziehung durch einen passenden Ansatz für $T(n)$. Die allgemeine Lösung ist dann die Summe

$$T(n) = T_h(n) + T_p(n) .$$

Aufgabe 8: Bestimmen Sie die Auflösung der folgenden Rekursionsbeziehungen:

a) $T(n) = 5 \cdot T(n-1)$ mit $T(0) = 4$. Grad 1

$$\left. \begin{array}{l} T(u) = a \cdot 5^u \\ T(0) = a \cdot 5^0 = 4 \\ a = 4 \end{array} \right\} T(u) = \underline{\underline{4 \cdot 5^u}}$$

b) $T(n) = 6 \cdot T(n-1) - 9 \cdot T(n-2)$ mit $T(0) = 1$ und $T(1) = 6$. Grad 2

$$\begin{array}{l} r^2 = 6r - 9 \\ r^2 - 6r + 9 = 0 \\ (r-3)(r-3) = 0 \\ r_1 = r_2 = 3 \end{array} \quad \begin{array}{l} T(u) = a \cdot 3^u + b \cdot u \cdot 3^u \\ T(0) = a \cdot 3^0 + b \cdot 0 \cdot 3^0 = 1 \\ T(1) = a \cdot 3^1 + b \cdot 1 \cdot 3^1 = 6 \\ \left. \begin{array}{l} a = 1 \\ 3a + 3b = 6 \end{array} \right\} \begin{array}{l} a = 1 \\ b = 1 \end{array} \end{array} \quad T(u) = \underline{\underline{3^u + u \cdot 3^u}} = \underline{\underline{(1+u) \cdot 3^u}}$$

c) $T(n) = T(n-1) + 2 \cdot T(n-2)$ mit $T(0) = 2$ und $T(1) = 7$. Grad 2

$$\begin{array}{l} r^2 = r + 2 \\ r^2 - r - 2 = 0 \\ (r+1)(r-2) = 0 \\ r_1 = 2, r_2 = -1 \end{array} \quad \begin{array}{l} T(u) = a \cdot 2^u + b \cdot (-1)^u \\ T(0) = a \cdot 2^0 + b \cdot (-1)^0 = 2 \\ T(1) = a \cdot 2^1 + b \cdot (-1)^1 = 7 \\ \left. \begin{array}{l} a + b = 2 \\ 2a - b = 7 \end{array} \right\} \begin{array}{l} a = 3 \\ b = -1 \end{array} \end{array} \quad T(u) = \underline{\underline{3 \cdot 2^u - (-1)^u}}$$

d) $T(n) = 3 \cdot T(n-1) + 2n$ mit $T(1) = 3$.
 Polynom vom Grad 1

$$\begin{array}{l} T_h(u) = a \cdot 3^u \\ \text{Ansatz: } T_p(u) = w \cdot u + b \\ T_p(u) - 3 \cdot T_p(u-1) = 2 \cdot u \\ w \cdot u + b - 3 \cdot (w \cdot (u-1) + b) = 2 \cdot u \\ \underline{-2w \cdot u + 3w - 2b} = \underline{2u + 0} \\ \underline{= 2} \quad \underline{= 0} \\ \text{also } w = -1 \quad \text{also } b = -1.5 \end{array} \quad \begin{array}{l} T(u) = T_h(u) + T_p(u) \\ = a \cdot 3^u + (-1) \cdot u + (-1.5) \\ = a \cdot 3^u - u - 1.5 \\ T(1) = a \cdot 3^1 - 1 - 1.5 = 3 \\ 3a = 5.5 \\ a = 11/6 \\ T(u) = \underline{\underline{\frac{11}{6} \cdot 3^u - u - 1.5}} \end{array}$$

e) $T(n) = 5 \cdot T(n-1) - 6 \cdot T(n-2) + 7^n$ mit $T(0) = 1$ und $T(1) = 15$.
 Exp-Funktion

$$\begin{array}{l} r^2 = 5r - 6 \\ r^2 - 5r + 6 = 0 \\ (r-3)(r-2) = 0 \\ r_1 = 2, r_2 = 3 \end{array}$$

$$T_h(u) = a \cdot 2^u + b \cdot 3^u$$

Ansatz: $T_p(u) = w \cdot 7^u + c$

$$\begin{array}{l} T_p(u) - 5 \cdot T_p(u-1) + 6 \cdot T_p(u-2) = 7^u \\ w \cdot 7^u + c - 5 \cdot (w \cdot 7^{u-1} + c) + 6 \cdot (w \cdot 7^{u-2} + c) = 7^u \quad \cdot 7^2 \\ w \cdot 7^2 \cdot 7^u + 7^2 c - 5 \cdot 7^2 w \cdot 7^{u-1} - 5 \cdot 7^2 c + 6 \cdot 7^2 w \cdot 7^{u-2} + 6 \cdot 7^2 c = 7^2 \cdot 7^u \\ 49w \cdot 7^u + 49c - 35w \cdot 7^u - 245c + 6w \cdot 7^u + 294c = 49 \cdot 7^u \end{array}$$

$$\underbrace{20m \cdot 7^n}_{=49} + \underbrace{38c}_{=0} = \underbrace{49 \cdot 7^n}_{=0} + \underbrace{0}_{=0}$$

also $m = \frac{49}{20}$ also $c = 0$

$$T(n) = T_h(n) + T_p(n)$$

$$= a \cdot 2^n + b \cdot 3^n + \frac{49}{20} \cdot 7^n$$

$$T(0) = a \cdot 2^0 + b \cdot 3^0 + \frac{49}{20} \cdot 7^0 = 1 \quad \} \cdot 20$$

$$T(1) = a \cdot 2^1 + b \cdot 3^1 + \frac{49}{20} \cdot 7^1 = 15 \quad \} \cdot 20$$

$$\left. \begin{array}{l} 20a + 20b = -29 \\ 40a + 60b = -43 \end{array} \right\} \begin{array}{l} a = -\frac{11}{5} \\ b = \frac{3}{4} \end{array}$$

$$T(n) = \underline{\underline{-\frac{11}{5} \cdot 2^n + \frac{3}{4} \cdot 3^n + \frac{49}{20} \cdot 7^n}}$$